



《人工智能综合设计》—爬虫设计与实现



人工智能综合设计 教研组



今日目标

- 实现爬虫
- 给定如下初始URL，爬取所有链接，保存所有的html文件
 - 建议存到文件里
 - 注意设置sleep
 - 爬取网站更换为 科研处+学工部
 - <http://keyan.ruc.edu.cn/> 、 <http://xsc.ruc.edu.cn/>



提纲



人工智能综合设计02 爬虫设计

- ☐ 使用Requests库获取http文档
- ☐ 爬虫设计和实现原理
-
-



什么是万维网

- 万维网的核心组成部分：
 - 统一资源标志符 (URI)
 - **Uniform Resource Identifier**
 - “网址”，例如：<https://www.ruc.edu.cn/>，<http://ai.ruc.edu.cn/>
 - 超文本标记语言 (HTML)
 - **HyperText Markup Language**
 - 用于创建网页的标准标记语言
 - 浏览器可以读取HTML文件，并将其渲染成可视化网页
 - 超文本传输协议 (HTTP)
 - **HyperText Transfer Protocol**
 - 服务器和浏览器通信，发布和接收HTML页面的协议
 - 通过HTTP协议请求的资源由统一资源标志符 (URI) 标识



浏览器是如何访问万维网的?

- 简单来说:
 - 浏览器解析URI, 向相应IP地址发送HTTP请求
 - 服务器解析HTTP请求, 返回包含HTML文档的HTTP响应
 - 浏览器解析HTML文档, 渲染网页
- 详细来说.....



浏览器是如何访问万维网的?

- 用户点击一个连接或者在地址栏输入一个URI后，浏览器会解析该URI：

`https://www.ruc.edu.cn/scienceandtechnology`

协议名称

- HTTPS：超文本传输**安全**协议

主机名称 (Host name)

- 通常是域名或者IP地址
- 域名会经过域名系统(Domain Name System, DNS)，转化为IP地址
 - 182.140.213.107
- IP地址：互联网中计算机的“地址”

路径 (Path)

- 定位具体的资源，例如一个HTML文档、一个图片



浏览器是如何访问万维网的?

- 浏览器会向服务器发起一个**HTTP请求 (request)**

```
1 GET /scienceandtechnology HTTP/1.1
2 Host: www.ruc.edu.cn
3 Connection: keep-alive
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like
7 Gecko) Chrome/86.0.4240.198 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
9 /*;q=0.8,application/signed-exchange;v=b3;q=0.9
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Accept-Encoding: gzip, deflate, br
14 Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,zh-TW;q=0.7
15 Cookie: ....
```




浏览器是如何访问万维网的?

- 浏览器会向服务器发起一个**HTTP请求 (request)**
 - 请求行: GET /scienceandtechnology HTTP/1.1
 - 请求方法: GET 获取相应路径制定的资源
 - 资源路径: /scienceandtechnology
 - 协议版本: HTTP/1.1
 - 常见的请求方法:
 - **GET: 获取相应路径制定的资源**
 - HEAD: 请求资源但只返回响应头 (response header)
 - POST: 向指定资源提交数据, 常常用于上传表单
 - PUT: 向指定资源上传最新内容
 - DELETE: 请求服务器删除指定资源



浏览器是如何访问万维网的?

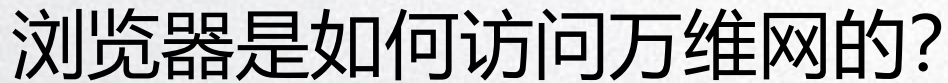
- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 响应头 (response header)

```
1 HTTP/1.1 200 OK
2 Date: Sat, 28 Nov 2020 04:54:45 GMT
3 Content-Type: text/html; charset=UTF-8
4 Link: <https://www.ruc.edu.cn/wp-json/>; rel="https://api.w.org/"
5 Vary: Accept-Encoding
6 Content-Encoding: gzip
7 Via: 3835
8 Cache-Control: max-age=600
```



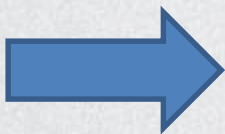
浏览器是如何访问万维网的？

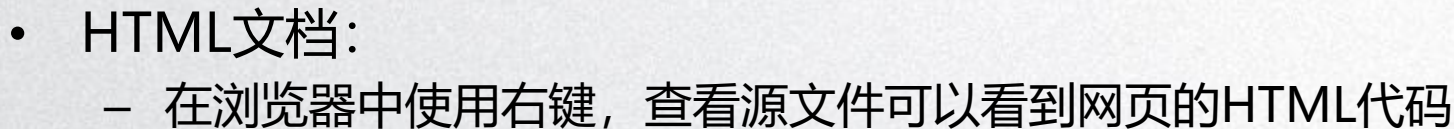
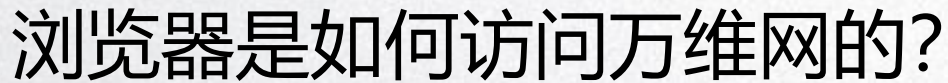
- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 状态行： HTTP/1.1 200 OK
 - HTTP状态码：
 - 1xx消息——请求已被服务器接收，继续处理
 - 2xx成功——请求已成功被服务器接收、理解、并接受
 - 200 OK 请求的资源已返回
 - 3xx重定向——需要后续操作才能完成这一请求
 - 302 Found 常用于将GET请求重定向到其他路径
 - 4xx请求错误——请求含有词法错误或者无法被执行
 - 403 Forbidden 服务器拒绝执行请求
 - 404 Not Found 服务器找不到请求的资源
 - 5xx服务器错误——服务器在处理某个正确请求时发生错误
 - 500 Internal Sever Error 服务器出错



- 服务器收到HTTP请求后，会对其进行处理，并返回**HTTP响应** (response)
 - 响应内容类别： Content-Type: text/html; charset=UTF-8
 - 响应内容： UTF-8编码的**HTML文档**
 - 浏览器会将HTML文档渲染成可视化的网页展示在屏幕上

```
<DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>人人网 | RENREN VISION OF CHINA</title>
<script>
    {
        host: 100%;
        // -webkit-filter: grayscale(100%);
        // -ms-filter: grayscale(100%);
        // -o-filter: grayscale(100%);
        // -ms-filter: grayscale(100%);
        // filter: grayscale(100%);
        // filter: gray;
    }
    h1{
        //filter: progid:DXImageTransform.Microsoft.BasicImage(grayscale=1);
        //filter: gray;
    }
</style>
<script type="text/javascript">
function sel()
{
    if (!i).load(window.location.href)
    window.location.href="/home324.html";
    return false;
}
</script>
if (window.innerWidth < 760){
    window.location.href="/mobile.html";
    return false;
}
if (window.innerWidth < 1280){
    window.location.href="/home324.html";
    return false;
}
</script>
<!--[if IE]--><script src="http://content.themenu.com/themes/rucw/js/jquery.laps.js" type="text/javascript"></script>
<!--[/if IE]>
<link rel="stylesheet" icon href="/wp-content/themes/rucw/laps/icon" type="image/icon"/>
<link rel="stylesheet" href="/wp-content/themes/rucw/css/style.css" />
<link rel="stylesheet" type="text/css" href="/wp-content/themes/rucw/css/btgrecher.css" />
<link rel="stylesheet" type="text/css" href="/wp-content/themes/rucw/css/style1.css" />
<script type="text/javascript" src="/apps.bing.com/lib/jquery/1.7.0/jquery.min.js"></script>
<script type="text/javascript" src="/wp-content/themes/rucw/js/btgrecher.js"></script>
```







使用Python语言实现上述过程

- 实现浏览器的功能：
 - 解析URI: urllib模块
 - 提交HTTP请求: **Requests库**
 - 解析HTML文档: **Beautifulsoup库**
- 实现服务器的功能：
 - 响应HTTP请求：
 - Python自带了一个简单的HTTP服务器, 可用于访问当前目录下的文件
 - 命令行输入: `python -m http.server 8000`
 - 浏览器访问: <http://localhost:8000> (或者<http://127.0.0.1:8000>)
 - 浏览器会自动渲染HTML文档, 并可以下载其他类型的文件
 - 如果目录下包含index.html文件, 服务器会默认返回该文件
 - 实现更为复杂的Web站点
 - 可使用Django、Pyramid、Flask等框架



Requests库

- Requests库是一个专门处理HTTP请求的第三方库
 - Requests: HTTP for Humans™
 - 优点：程序编写过程更接近正常URI访问过程。
 - Requests库的文档请见：<https://requests.readthedocs.io/>
 - 安装方法：
 - 命令行输入：pip install requests



使用Requests库请求网页

- 构造和发送请求
 - 可以使用一个dict指定HTTP请求头中的信息
 - requests.get对应于HTTP请求方法中的GET方法，执行该函数就会发送一个相应的HTTP请求

```
[40]: # 请求网页
import requests

uri = "https://www.ruc.edu.cn/scienceandtechnology" # 要访问的资源的URI
headers = {'user-agent': 'my-app/0.0.1'} # 中国人民大学的网站会限制请求头里不包含user-agent信息的请求
r = requests.get(uri, headers=headers) # 使用GET方法请求URI指定的资源，并且在请求头中添加headers中信息
```



使用Requests库请求网页

- 构造和发送请求
 - 使用其他函数可以发送对应于不同HTTP请求方法的请求，如：

HTTP请求方法	Requests库提供函数
GET	requests.get
HEAD	requests.head
POST	requests.post
PUT	requests.put
DELETE	requests.delete



使用Requests库请求网页

- 处理响应

- requests.get函数会返回一个对应于HTTP请求的Response对象

```
print(type(r)) # 获得一个对应于HTTP响应的Response对象
```

```
print("Status code:", r.status_code) # 响应的状态代码
```

```
print("Response header:", r.headers) # 响应头信息
```

```
<class 'requests.models.Response'>
```

```
Status code: 200
```

```
Response header: {'Date': 'Sat, 28 Nov 2020 17:42:56 GMT', 'Content-Type': 'text/html; charset=UTF-8', 'Transfer-Encoding': 'chunked', 'Link': '<https://www.ruc.edu.cn/wp-json/>; rel="http://api.w.org/"', '<https://www.ruc.edu.cn/?p=16627>; rel=shortlink', 'Vary': 'Accept-Encoding', 'Content-Encoding': 'gzip', 'Via': '3835', 'Cache-Control': 'max-age=600'}
```

- 需要注意:

- 出现网络连接错误、链接超时等错误时执行requests.get会抛出异常
- 但成功连接服务器不意味着请求成功的完成 (不一定是200 OK)



使用Requests库请求网页

- 处理响应

- 需要注意:

- 出现网络连接错误、链接超时等错误时执行requests.get会抛出异常
 - 但成功连接服务器不意味着请求成功的完成 (不一定是200 OK)

```
[49]: # 需要注意: 出现网络连接错误、链接超时等错误时执行requests.get会抛出异常
r = requests.get("http://this_domain_does_not_exist.com")
```

```
ConnectionError: HTTPConnectionPool(host='this_domain_does_not_exist.com', port=80): Max retrie
s exceeded with url: / (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object
at 0x10b49fbd0>: Failed to establish a new connection: [Errno 8] nodename nor servname provide
d, or not known'))
```

```
[51]: # 但成功连接服务器不意味着请求成功的完成
r = requests.get("http://www.ruc.edu.cn/path_to_nowhere", headers=headers)
print(r.status_code)
```

```
404
```

- 可以使用r.raise_for_status()在请求不成功时抛出异常



使用Requests库请求网页

- 处理响应：
 - 可以使用r.content访问响应二进制形式的内容
 - 在r.encoding编码设置正确的情况下，可以使用r.text访问响应的文本形式的内容
 - 编码决定程序如何用二进制数据表示文本数据
 - 常见的编码：
 - ASCII：用一个字节（Byte，八位二进制数）编码数字、大小写英文字母和常见英文符号
 - UTF-8：万维网的最主要的编码形式（在所有网页中，应用率高达94.3%）
 - GBK, GB18030：中文编码
 - 默认的编码根据响应头中信息指定：
 - Content-Type: text/html; charset=UTF-8
 - 如果编码设置错误，r.text会出现乱码问题，可通过给r.encoding赋值指定编码

使用Requests库请求网页

- 一个更好的获取HTML文档的函数

```
[52]: #一个更为鲁棒 (robust) 的获取HTML文档的函数
def get_html(uri, headers={}, timeout=30):
    try:
        r = requests.get(uri, headers=headers, timeout=timeout)
        r.raise_for_status()
        return r.text
    except:
        return None

uri = "https://www.ruc.edu.cn/scienceandtechnology" # 要访问的资源的URI
headers = {'user-agent': 'my-app/0.0.1'}
print(get_html(uri, headers=headers))
```


一个简单的抓取万维网信息的例子

```
[116]: # 一个完整的从真实网页上获取信息的例子
uri = "https://www.ruc.edu.cn/schools-az" # 要访问的资源的URI
headers = {'user-agent': 'my-app/0.0.1'}
html_doc = get_html(uri, headers=headers, timeout=None)
soup = BeautifulSoup(html_doc)

data = {} # 新建字典保存信息
for dep_category in soup.find_all('div', class_="dcm-content-dep-cat"): #遍历学部
    # 获取学部名称
    dep_category_tag = dep_category.find('div', class_="dcmc-dep-cat-name")
    dep_category_name = dep_category_tag.contents[-1].strip()

    schools = {} # 新建字典保存保存学部下学院信息
    for school in dep_category.find_all('div', class_="dcmc-dep-cat-list-ins"): #遍历学院
        # 获取学院名称
        school_name = school.find('div', class_="dcmc-dep-cat-list-ins-name").string
        # 获取学院官网地址
        schools[school_name] = [a['href'] for a in school.find_all('a')]

    data[dep_category_name] = schools # 将学院信息保存到按学部索引的字典里
```



一个简单的抓取万维网信息的例子

```
: print(data)
```

```
{ '人文': { '哲学院': ['http://phi.ruc.edu.cn'], '文学院': ['http://wenxueyuan.ruc.edu.cn'], '历史学院': ['http://lsxy.ruc.edu.cn/'], '国学院': ['http://guoxue.ruc.edu.cn/'], '艺术学院': ['http://art.ruc.edu.cn'], '外国语学院': ['http://fl.ruc.edu.cn/'], '清史研究所': ['http://www.iqh.net.cn'], '国际文化交流学院': ['http://scsce.ruc.edu.cn']}, '社会': { '新闻学院': ['http://jcr.ruc.edu.cn/'], '农业与农村发展学院': ['http://www.sard.ruc.edu.cn/'], '社会与人口学院': ['http://ssps.ruc.edu.cn/'], '公共管理学院': ['http://spap.ruc.edu.cn/'], '信息资源管理学院': ['http://irm.ruc.edu.cn/'], '教育学院': ['http://soe.ruc.edu.cn']}, '经济': { '经济学院': ['http://econ.ruc.edu.cn/'], '财政金融学院': ['http://sf.ruc.edu.cn/'], '统计学院': ['http://stat.ruc.edu.cn'], '商学院': ['http://www.rmb.s.ruc.edu.cn'], '劳动人事学院': ['http://slhr.ruc.edu.cn/'], '中法学院': ['http://sc.ruc.edu.cn'], '汉青经济与金融高级研究院': ['http://www.hanqing.ruc.edu.cn'], '中国经济改革与发展研究院': ['http://www.yjy.ruc.edu.cn'], '中国人民大学国家发展与战略研究院': ['http://NADS.ruc.edu.cn'], '国际学院 (苏州研究院)': ['http://sc.ruc.edu.cn'], '统计与大数据研究院': ['http://isbd.ruc.edu.cn'], '重阳金融研究院': ['http://www.rdcy.org']}, '法政': { '法学院': ['http://www.law.ruc.edu.cn'], '马克思主义学院': ['http://marx.ruc.edu.cn'], '国际关系学院': ['http://sis.ruc.edu.cn'], '知识产权学院': ['http://ipr.ruc.edu.cn'], '律师学院': ['http://lawyer.ruc.edu.cn/'], '亚太法学研究院': ['http://apil.ruc.edu.cn/'], '丝路学院': ['http://srs.ruc.edu.cn/']}, '理工': { '环境学院': ['http://envi.ruc.edu.cn'], '信息学院': ['http://info.ruc.edu.cn'], '理学院': ['http://www.phys.ruc.edu.cn/'], 'http://chem.ruc.edu.cn/', 'http://psy.ruc.edu.cn/'], '数学学院': ['http://math.ruc.edu.cn/'], '高瓴人工智能学院': ['http://ai.ruc.edu.cn']}, '其他': { '继续教育学院': ['http://sce.ruc.edu.cn/'], '国际教育学院': ['http://sie.ruc.edu.cn/'], '网上人大': ['http://www.cmr.com.cn/'], '深圳研究院': ['http://sz.ruc.edu.cn'] }}
```



提纲



人工智能综合设计02 爬虫设计

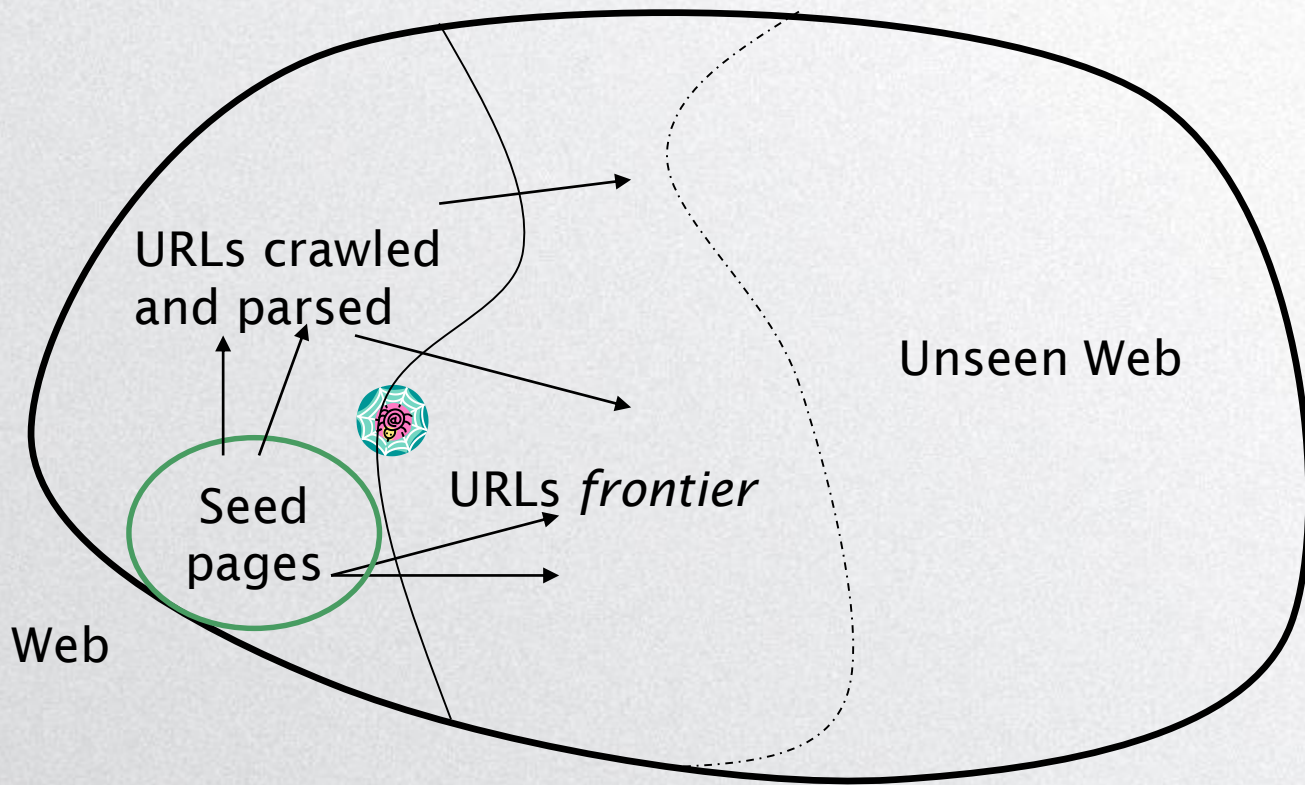
- ☐ 使用Requests库获取http文档
- ☐ 网络爬虫设计和实现原理
-
-



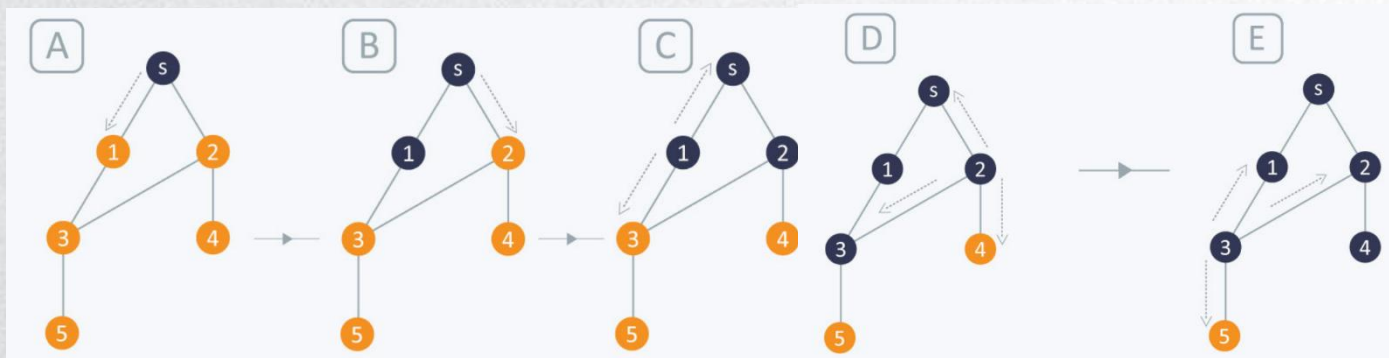
基本爬虫操作

- 从已知的“种子”URLs开始
- 抓取并解析它们
 - 抓取它们指向（包含）的URLs
 - 将抓取的URLs放入队列
- 获取队列中的每个URL，重复以上操作

爬虫过程



广度优先搜索(BFS)





广度优先搜索(BFS)

1. 首先将根节点放入队列中。
2. 从队列中取出第一个节点，并检验它是否为目标。
 - 如果找到目标，则结束搜索并回传结果。
 - 否则将它所有尚未检验过的直接子节点加入队列中。
3. 若队列为空，表示整张图都检查过了——亦即图中没有欲搜索的目标。结束搜索并回传“找不到目标”。
4. 重复步骤2。



广度优先遍历

- 维护一个队列，首先将初状态加入队列中，标记该状态已搜索，然后：
 - 1) 取出队首元素，将它所有可产生的未标记后继状态加入队列，并将其标记为已搜索；
 - 2) 当队列未空时重复1)；

从简单到复杂

- 一台机器无法抓取大量网页
 - 并行/分布式抓取
- 恶意页面
 - 垃圾页面
 - 蜘蛛陷阱 – 动态生成
- 即使是非恶意页面也会带来挑战
 - 远程服务器的延迟/带宽各不相同
 - 网站管理员的规定
 - 爬虫可以抓取网站的 URL 层次结构有多“深”??
 - 网站镜像和重复页面
- 礼貌——不要太频繁地访问服务器

爬虫必须具有的性质

- 鲁棒 (robustness)：不受网络服务器的蜘蛛陷阱和其他恶意行为的影响
- 礼貌 (Politeness)：尊重隐含和明确的礼貌考虑

显性和隐性礼貌

- 明确的礼貌：网站管理员对可以抓取网站的哪些部分的规定
 - robots.txt
- 含蓄的礼貌：即使没有规范，也避免过于频繁地访问任何网站

Robots.txt

- 允许蜘蛛（“robots”）有限访问网站的协议，最初出现在1994年
 - www.robotstxt.org/robotstxt.html
- 网站宣布其对可以（或不能）被抓取的内容的要求
 - 对于服务器，创建一个文件 `/robots.txt`
 - 此文件指定访问限制

Robots.txt 样例

- 爬虫不应访问任何以 “/yoursite/temp/” 开头的 URL，除了名为 “searchengine” 的爬虫

```
User-agent: *
```

```
Disallow: /yoursite/temp/
```

```
User-agent: searchengine
```

```
Disallow:
```

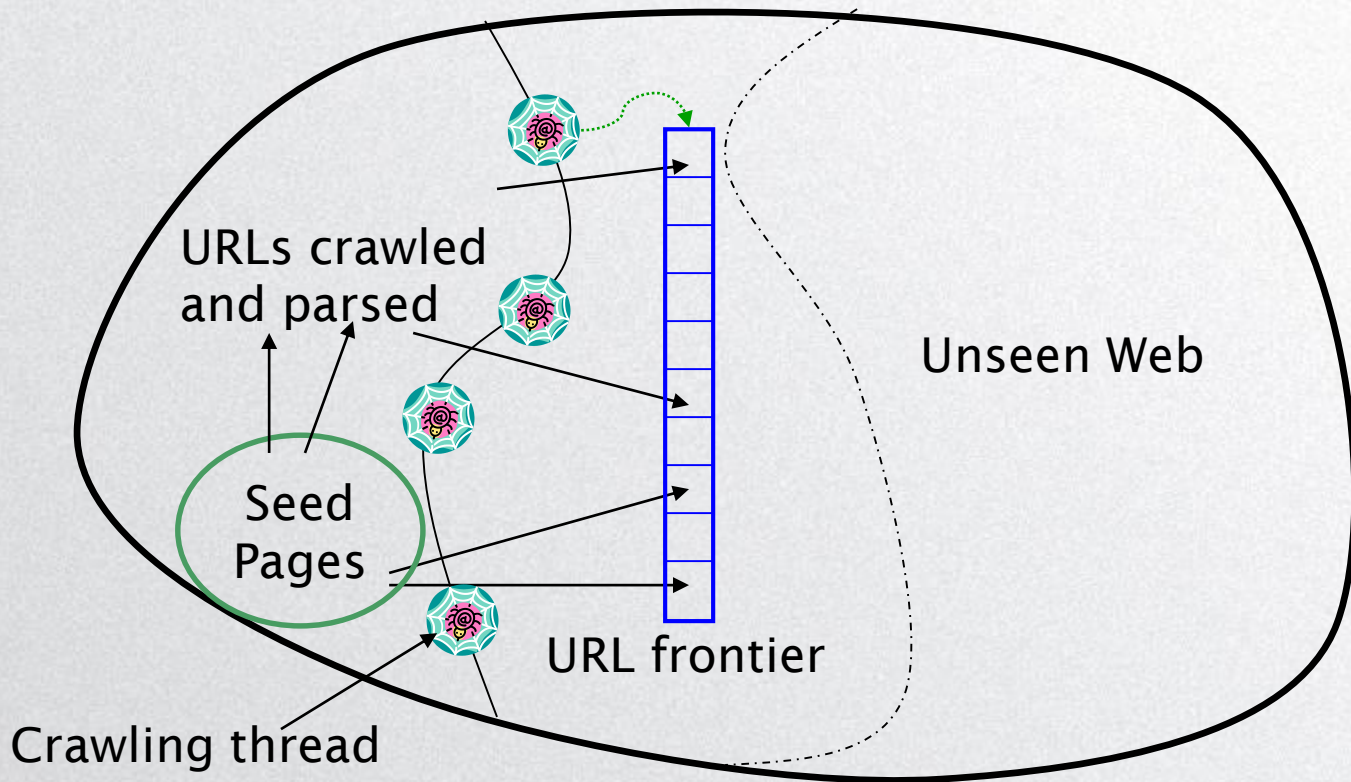
爬虫应具有的能力

- 能够分布式运行：设计运行在多台分布式机器上
- 可扩展：旨在通过使用更多机器来提高抓取
- 速度性能/效率：允许充分利用可用的处理和网络资源

爬虫应具有的能力

- 首先抓取 “更高质量” 的页面
- 连续操作：继续抓取先前获取页面的新副本
- 可拓展：适应新的数据格式、协议

更新爬虫过程





URL 边界

- 可以包含来自同一host的多个页面
- 必须避免试图同时抓取它们
- 必须尽量让所有爬行线程保持忙碌

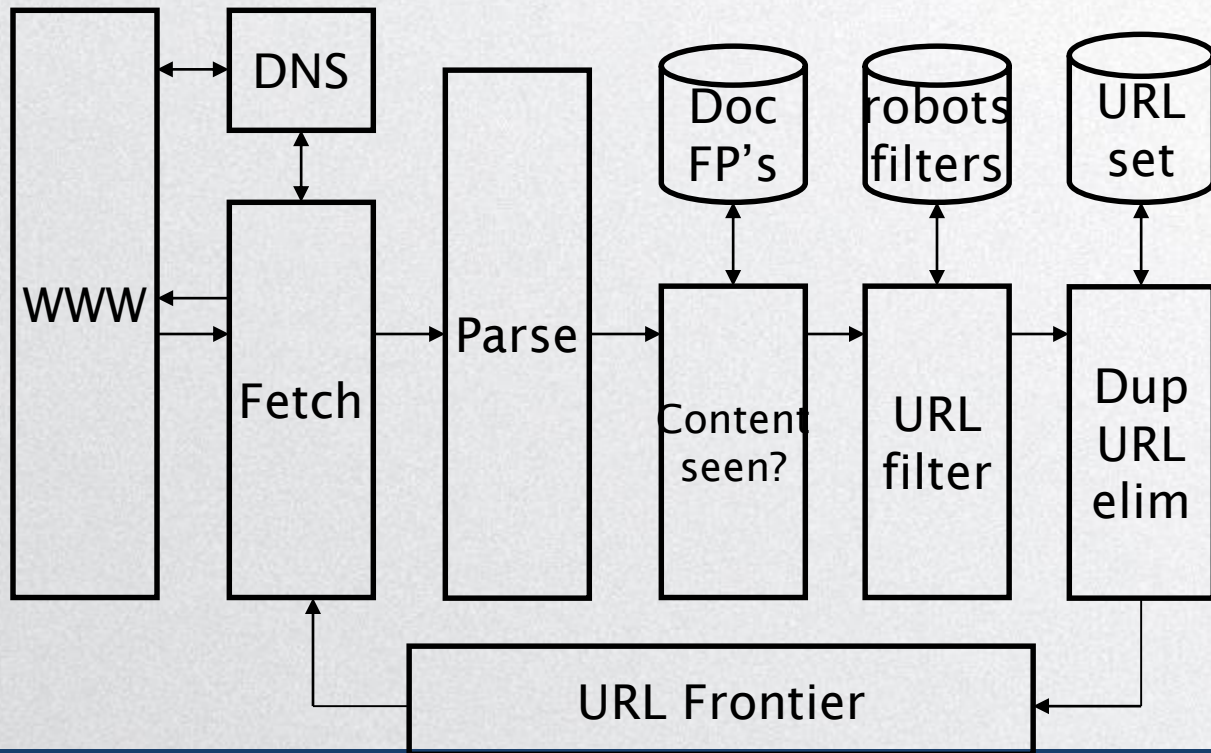
爬虫处理步骤

- 从边界中选择一个 URL
- 获取 URL 处的文档
- 解析URL
 - 从中提取链接到的其它文档 (URL)
 - 检查 URL 中的内容是否已经看到过
 - 如果没有, 添加到索引
- 对每一个提取的URL
 - 确保它通过某些 URL 过滤器测试
 - 检查是否已经在边界中 (消除重复URL)

哪一个?

比如只爬.edu, 服从 robots.txt等。

基本爬取架构





DNS (域名服务器)

- 互联网上的查找服务
 - 给定一个 URL，检索其 IP 地址
 - 由一组分布式服务器提供的服务——因此，查找延迟可能很高（甚至几秒）
 - DNS 查找的常见操作系统实现是阻塞的：一次只有一个未完成的请求
- 解决方案
 - DNS缓存
 - 批量 DNS 解析器 - 收集请求并将它们一起发送出去

解析：URL 规范化

- 当解析获取的文档时，一些提取的链接是相对 URL
 - 例如，http://en.wikipedia.org/wiki/Main_Page 有一个到 /wiki/Wikipedia:General_disclaimer 的相对链接，它与绝对 URL 路径 http://en.wikipedia.org/wiki/Wikipedia:General_disclaimer 相同
- 在解析时，必须规范化（扩展）这样的相对 URL



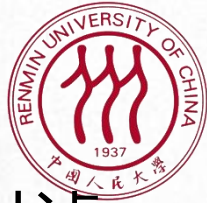
内容已经见到过?

- 重复在网络上很普遍
- 如果刚刚抓取的页面已经在索引中，则不要进一步处理它
- 使用文件fingerprints 或 shingles验证



过滤器和 robots.txt

- 过滤器 - 要抓取/不抓取的 URL 的正则表达式
- 从站点抓取 robots.txt 文件后，无需重复抓取
- 这样做会消耗带宽，影响网络服务器
- 缓存 robots.txt 文件



重复URL消除

- 对于非连续（一次性）抓取，测试是否已将提取+过滤的 URL 传递到边界
- 对于连续抓取 - 查看边界实现的详细信息



URL 边界：两个主要考虑因素

- 礼貌：不要太频繁地访问网络服务器
- 新鲜度：比其他页面更频繁地抓取某些页面
 - 例如，内容经常变化的页面（如新闻网站）
- 这些目标可能相互冲突。（例如，简单的优先级队列失败——页面的许多链接都转到其自己的站点，从而产生对该站点的大量访问。）

礼貌——挑战

- 即使我们限制只有一个线程从主机获取，也可以重复命中它
- 常见启发式：在对一个主机的连续请求之间插入时间间隔，即 $> \Delta t$ 最近从该主机抓取的时间



重复文档

- 网络上充满了重复的内容
- 严格重复检测 = 精确匹配
 - 不常见
- 但是很多很多几乎重复的案例
 - 例如，最后修改日期是页面的两个副本之间的唯一差异



重复/接近重复检测

- *重复 (Duplication)* : 可以用fingerprints检测完全匹配
- *接近重复 (Near-Duplication)* : 近似匹配
 - 概述
 - 使用编辑距离度量计算句法相似度
 - 使用相似度阈值检测近似重复
 - 例如, 相似度 > 80% => 文档 “接近重复”
 - Not transitive though sometimes used transitively



总体流程



- 1. 将指定的种子URLs放到队列里
- 2. 依次从队列中弹出一个URL，进行如下处理，直到队列为空
 - 抽取该URL文档中的正文和所有URLs，将抽取的html文档保存到本地，本地文件和其URL一一对应
 - 新抽取的URLs做重复检测，将不重复的URL放入到队列中

注意：抽取URL时加入sleep，否则过于频繁访问会被封锁IP



爬虫实现



```
# 导入库
from urllib.request import urljoin
from bs4 import BeautifulSoup
import requests
from urllib.request import urlparse
import time
from url_normalize import url_normalize

def crawl_all_urls(html_doc, url):
    all_links = set()
    try:
        soup = BeautifulSoup(html_doc, 'html.parser')
    except:
        print("Fail to parse the html document!")
        return all_links
    for anchor in soup.find_all('a'):
        href = anchor.attrs.get("href")
        if (href != "" and href != None):
            if not href.startswith('http'):
                href = url + href
            all_links.add(url_normalize(href))
    return all_links
```



爬虫实现

```
# 从给定的url中获取html文档
def get_html(uri, headers={}, timeout=None):
    try:
        r = requests.get(uri, headers=headers, timeout=timeout)
        r.raise_for_status()
        r.encoding='UTF-8'
        return r.text
    except:
        return None

headers = {'user-agent': 'my-app/0.0.1'}

# 初始化队列
queue = []
# 全局urls集合
all_urlset = set()
for url in input_urls:
    if url not in all_urlset:
        queue.append(url)
        all_urlset.add(url)
```

爬虫实现



```
count = 0
wait_time = 5
# htmlpath = r"C:\Bing\"
used_urlset = set()
while (len(queue)>0 and count<5):#此处仅迭代5步
    count = count + 1
    url = queue.pop(0) # 弹出队前一个url
    used_urlset.add(url)
    html_doc = get_html(url, headers = headers) # 从给定的url中获取html文档
    if html_doc is None:
        continue
    url_sets = crawl_all_urls(html_doc, url) # 第一天实现的方法 crawl_all_urls: 输入一个html文档, 返回所有urls
    # print(url_sets)
    for new_url in url_sets:
        # is_duplicated = judge_duplicated(new_url, all_urlset) #判断新url是否重复
        if new_url not in all_urlset:
            queue.append(new_url) #若不重复, 加入队列
            all_urlset.add(new_url)

    if wait_time > 0:
        print("等待 {} 秒后开始抓取".format(wait_time))
        time.sleep(wait_time)
```




爬虫实现

- 建议保存访问过的URL和html文件
- 注意命名

```
#      #保存当前html_doc, 防止被封锁
#      path = htmlpath + str(count) + ".html"
#      with open(path, 'w', encoding='utf-8') as f:
#          f.write(html_doc)
#          f.close()
```



谢谢！